Iteration Report

Iteration #1

Objectives:

- **Team Objectives/Project Scope:**
  - Create a C++ program that compares a classical vs. quantum search algorithms
    - Each algorithm should find a single item in an unsorted list of a certain size N
    - Program returns query, iteration, and success data
- **Individual Objectives:**
  - **Savannah:**
    - set up project github repo
    - implement linear and binary search, build and test classical search
    - implement query and time counting for all algorithms
  - **Amanda:**
    - Simulate quantum computing through Grover's algorithm
    - Test search algorithms on varying sets and sizes of data

Technologies and Tools:

- C++
  - the programming language for this project
- VSCode
  - an IDE will allow us to edit, organize, compile, and debug code in one interface
- Github
  - Store code, track changes, and work collaboratively
- Timeline and Planning documentation
  - Keep track of deadlines, visualize goals, measure progress

Project Timeline:

Milestone	Timeline	Task	Deliverable	Team Member	Deadline
Understand concepts of quantum searching	October	Research Grover's algorithm and basics of quantum computing	Planning doc of prospective design ideas	Amanda and Savannah	October 31
Agree on program design	November W1	Set up GitHub Repo	GitHub repo with README, folders, .gitignore	Savannah	November 6

		Create program class design	Diagram of program design with classes	Amanda	November 6
Classical search implemented	November W2	Implement linear and binary search algorithms	Search classes pass tests	Savannah	November 12
		Implement breadth-first and depth-first search algorithms	Search classes pass tests	Amanda	November 12
		Write test classes	Test class for each algorithm using dataset of integers	Amanda and Savannah	November 16
Quantum search implemented	November W3-4	Implement grover's algorithm	Search class passes tests	Savannah and Amanda	November 20
		Write test class	Test class algorithm using dataset of integers	Amanda	November 24
Working demonstration of search comparisons	December W1	Implement query and time counting for all algorithms	Program returns query, iteration, and time data for searches	Savannah	December 4
		Implement comparison driver	Program returns table of data comparing algorithm queries and success	Amanda	December 4
Final report and presentation completed	December W2	Create presentation slides	Complete slide deck	Amanda and Savannah	December 11
		Write Final Report	Complete report	Amanda and Savannah	December 11

- **Communication, Progress Tracking, and Addressing Challenges**

- Communication: Chat, Online Meeting, In-Person Meeting
- Progress Tracking: Review and Update Document, Review each request on git
- Addressing Challenges: Ask one another for help if needed. Look for TA or Professor help if necessary

Contribution Table:

	Savannah	**Amanda**
Iteration #1	- Helped form document with objectives, plan, tools, and communication details	- created timeline, contributed to objectives, scope, and technologies and tools
Iteration #2	- research and understand topics of quantum searching	

Iteration #2

Define All Task

- completed in excel project plan

Design the System

Classes

Class	Private data	Public methods
SearchResult (struct)	foundIndex, queries, iterations, timeMicros, success, successProb	(plain data, no methods)
SearchAlgorithm (abstract base)	name	virtual SearchResult search(const std::vector<int>& data, int target) = 0, getName()
LinearSearch : SearchAlgorithm	(none)	search()
BinarySearch : SearchAlgorithm	sortQueries	search(), getSortQueries()
BFSSearch : SearchAlgorithm	(none)	search() (uses std::queue)
DFSSearch : SearchAlgorithm	(none)	search() (uses std::stack)
QuantumState	std::vector<double> amplitudes, numQubits	initUniform(), applyOracle(int target), applyDiffusion(), probability(int i), measure()
GroverSearch : SearchAlgorithm	QuantumState state, random engine	search(), optimalIterations(int N)

Timer	start, end (std::chrono)	start(), stop(), elapsedMicros()
DataGenerator	random engine, seed	generateUnsorted(int N), pickTarget(const std::vector<int>&)
ComparisonDriver	std::vector<std::unique_ptr<SearchAlgorithm>> algorithms	addAlgorithm(), runAll(std::vector<int> sizes, int trials), printTable()

File Structure

```
├── README.md
├── Makefile (or CMakeLists.txt)
├── docs/          ← Iteration PDFs + Excel plan
├── include/
│   ├── search_result.h      search_algorithm.h
│   ├── linear_search.h      binary_search.h
│   ├── bfs_search.h         dfs_search.h
│   ├── quantum_state.h      grover_search.h
│   ├── timer.h              data_generator.h
│   ├── comparison_driver.h  tasks.h   tests.h
└── src/
    ├── main.cpp   tasks.cpp   tests.cpp
    └── (one .cpp per header above)
```

Diagram of Classes & How They Connect

